

Java Backend Architecture

Interview Series

Chapter 13.8

Memory Profiling & GC Tuning

50+ Interview Q&A

Code Examples

Architecture Diagrams

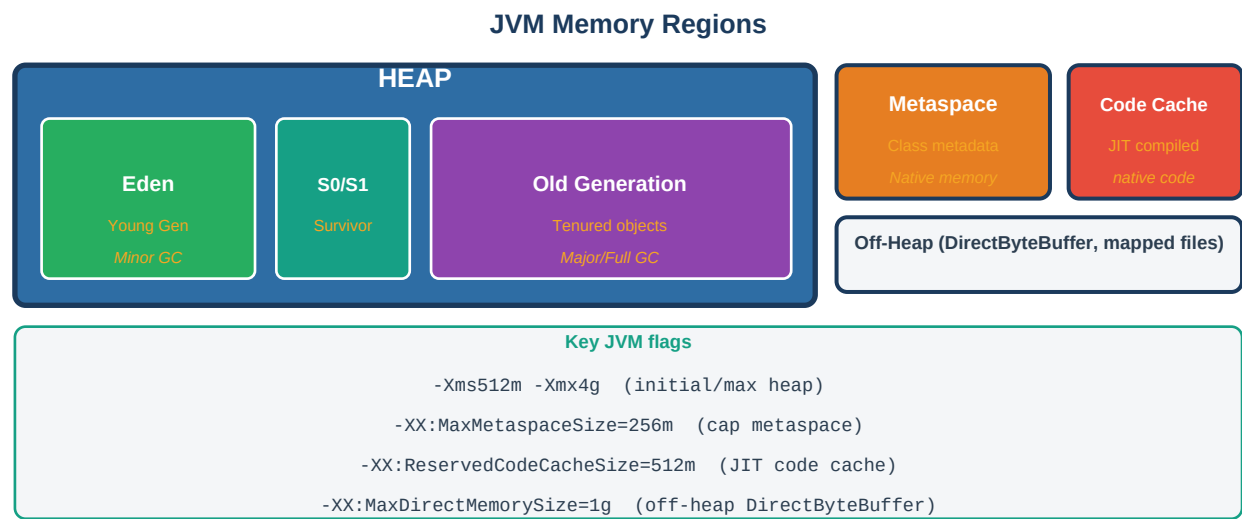
Advanced Topics

Memory Profiling and GC Tuning

JVM memory management is one of the most consequential aspects of Java backend performance. The JVM heap is divided into generations (Young: Eden and Survivor spaces; Old Generation) managed by a garbage collector. Beyond the heap, Metaspace holds class metadata, the Code Cache holds JIT-compiled native code, and off-heap memory is accessible via DirectByteBuffer. Understanding where objects live and how they are collected is essential for diagnosing OutOfMemoryError, excessive GC pauses, and memory leaks — all of which directly impact latency and availability in production systems.

GC algorithm selection significantly affects pause characteristics: G1GC (default since JDK 9) targets configurable pause goals via -XX:MaxGCPauseMillis; ZGC and Shenandoah deliver sub-millisecond pauses by doing most work concurrently with the application. GC log analysis tools (GCEasy, GCViewer) parse verbose GC output to identify pause patterns and tuning opportunities. Heap dump analysis using Eclipse MAT reveals object retention graphs, dominator trees, and the top objects by retained size — the foundation of memory leak diagnosis. Weak, soft, and phantom references provide escape hatches from strong reference retention.

JVM Memory Regions

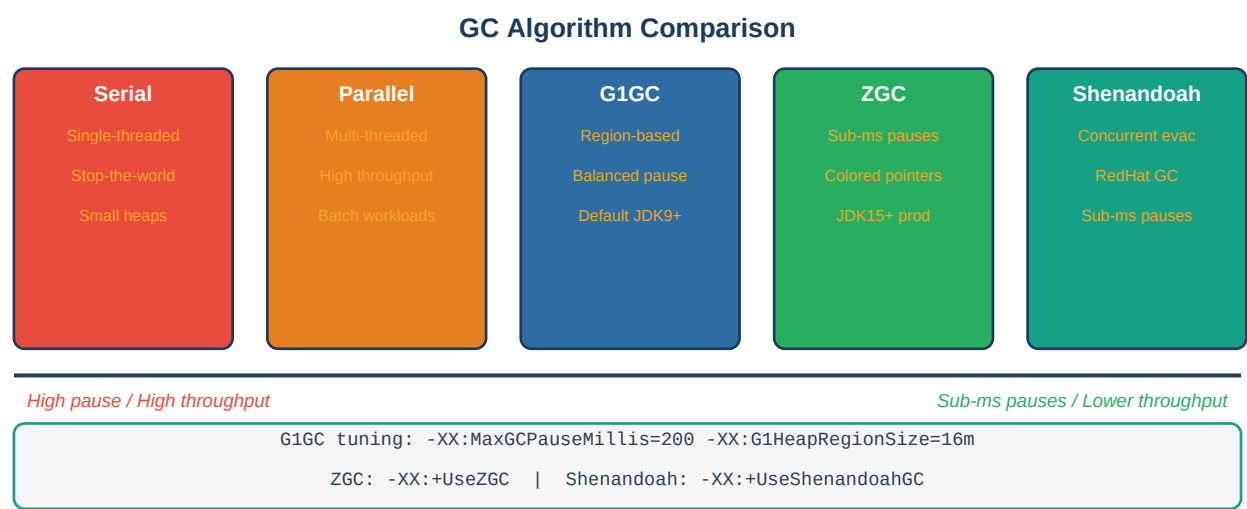


Key Definitions

Eden Space	Region of Young Generation where new objects are allocated via bump-pointer. Collected by Minor (Young) GC.
Survivor Spaces	Two equal-sized regions (S0/S1) in Young Gen. Objects surviving Minor GC are copied here and age-counted.
Old Generation	Region holding tenured objects (survived enough Minor GCs). Collected by Major GC (G1 concurrent) or Full GC.
Metaspace	Native memory region (not on heap, since JDK 8) holding class metadata, interned strings, and static fields.
Code Cache	Native memory region holding JIT-compiled native code. Exhaustion causes JIT to fall back to interpretation.

Off-Heap Memory	Memory outside the JVM heap, accessed via DirectByteBuffer. Not subject to GC; must be released explicitly.
G1GC	Garbage-First GC (default JDK 9+). Divides heap into equal-sized regions; targets MaxGCPauseMillis goal.
ZGC	Low-latency GC (JDK 15+ production-ready) using colored pointers and load barriers for sub-millisecond pauses.
Shenandoah	Concurrent evacuation GC by Red Hat, similar goals to ZGC. Available in OpenJDK, delivers sub-millisecond pauses.
GCEasy / GCViewer	Tools that parse JVM GC log files (-Xlog:gc) to visualise pause times, GC frequency, and heap utilisation trends.
Eclipse MAT	Memory Analyzer Tool for Eclipse. Opens HPROF heap dumps, runs Leak Suspects report and Dominator Tree analysis.
Dominator Tree	In heap analysis, the dominator of object X is the nearest ancestor that must be retained to keep X alive. Tree shows retained sizes.
Retained Size	The amount of heap freed if an object and all objects exclusively reachable through it were garbage-collected.
Weak Reference	java.lang.ref.WeakReference. Allows GC to collect the referent in the next GC cycle. Used in caches (WeakHashMap).
Soft Reference	java.lang.ref.SoftReference. JVM clears soft references before throwing OutOfMemoryError. Used for memory-sensitive caches.

GC Algorithm Comparison



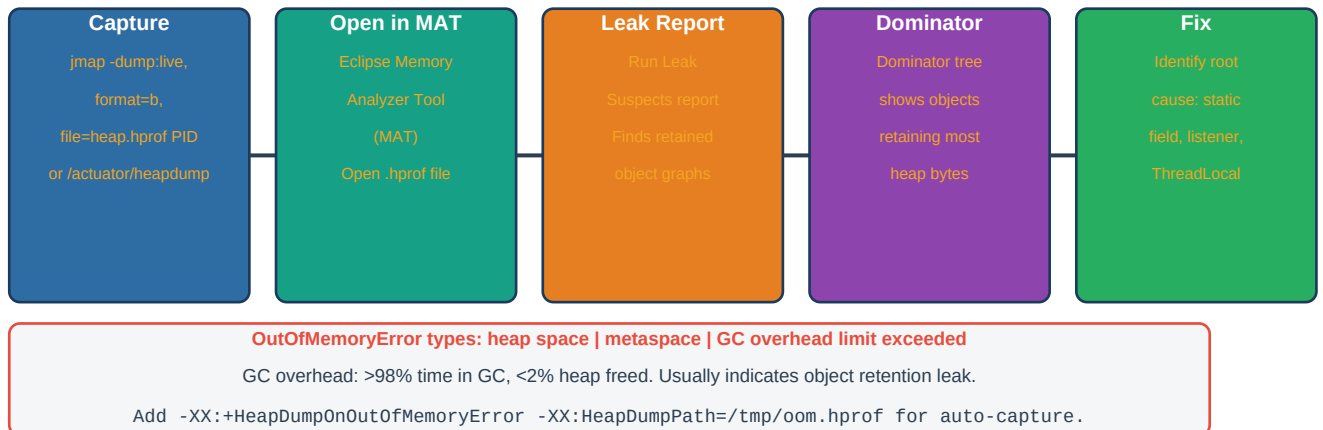
GC Algorithm Quick Reference

GC Algorithm	JVM Flag	Pause Type	Pause Target	Best For
Serial	-XX:+UseSerialGC	Stop-the-world	None	Single-core, tiny heaps (<100MB)
Parallel	-XX:+UseParallelGC	Stop-the-world	None	Batch throughput, offline jobs

G1GC	-XX:+UseG1GC (default)	Mostly concurrent	200ms	General-purpose, heap 4GB-64GB
ZGC	-XX:+UseZGC	Sub-millisecond	<1ms	Low-latency, large heaps (TB)
Shenandoah	-XX:+UseShenandoahGC	Sub-millisecond	<1ms	Low-latency, OpenJDK

Heap Dump Analysis Workflow

Heap Dump Analysis Workflow



Annotated Code Examples

1. GC logging and G1GC tuning flags

```
# JDK 11+ unified logging format -Xms2g -Xmx8g -XX:+UseG1GC -XX:MaxGCPauseMillis=200
-XX:G1HeapRegionSize=16m -XX:G1NewSizePercent=20 -XX:G1MaxNewSizePercent=40
-XX:InitiatingHeapOccupancyPercent=45 # Enable GC logging
-Xlog:gc*,gc+phases=debug:file=/tmp/gc.log:time,uptime,level,tags:filecount=5,filesize=20m #
Heap dump on OOM -XX:+HeapDumpOnOutOfMemoryError -XX:HeapDumpPath=/tmp/oom.hprof # ZGC (JDK 15+
production) -XX:+UseZGC -XX:SoftMaxHeapSize=6g # ZGC tries to keep live set below this -Xmx8g
```

2. Heap dump capture and analysis with jmap + MAT

```
# Capture heap dump (live=exclude unreachable objects, smaller file) jmap
-dump:live,format=b,file=/tmp/heap.hprof <PID> # Via Spring Boot Actuator (POST request
triggers dump download) curl -X GET http://localhost:8080/actuator/heapdump -o /tmp/heap.hprof
# Eclipse MAT: open heap.hprof # 1. Run: File -> Open Heap Dump # 2. Run: Reports -> Leak
Suspects Report # -> Shows top suspect objects with retained size # 3. Open: Window -> Heap
Dump Overview -> Biggest Objects # 4. OQL query to find all instances of a class: # SELECT *
FROM java.util.HashMap # SELECT * FROM com.example.SessionData # jmap histogram - quick top
classes by instance count jmap -histo:live <PID> | head -30
```

3. Object retention patterns — memory leak examples

```
// LEAK PATTERN 1: Static field holding instance references public class EventBus { // Static
list grows forever if listeners are never removed private static final List<EventListener>
listeners = new ArrayList<>(); public static void register(EventListener l) { listeners.add(l);
} // FIX: provide unregister() or use WeakReference } // LEAK PATTERN 2: Inner class holding
outer class reference public class OuterService { private LargeData data = new LargeData(); //
Non-static inner class implicitly holds reference to OuterService // If InnerTask is stored in
a long-lived collection, OuterService leaks class InnerTask implements Runnable { /* implicitly
holds OuterService.this */ } // FIX: make InnerTask static (or separate top-level class) }
```

```
LEAK PATTERN 3: ThreadLocal not cleaned up private static final ThreadLocal<LargeObject>
threadLocal = new ThreadLocal<>(); public void process() { threadLocal.set(new LargeObject());
try { // ... work ... } finally { threadLocal.remove(); // REQUIRED – thread pools reuse
threads indefinitely } }
```

4. Weak, Soft, and Phantom References

```
import java.lang.ref.*; // WeakReference – collected at next GC cycle WeakReference<Cache>
weakCache = new WeakReference<>(new Cache()); Cache c = weakCache.get(); // returns null after
GC collects it // WeakHashMap – entries auto-removed when key is GC-collected Map<Widget,
Metadata> map = new WeakHashMap<>(); // SoftReference – collected only when heap is nearly full
(before OOM) SoftReference<Image> softImg = new SoftReference<>(loadImage("logo.png")); Image
img = softImg.get(); // may return null if JVM needed to reclaim if (img == null) img =
loadImage("logo.png"); // reload if cleared // PhantomReference – used for post-finalization
cleanup ReferenceQueue<Resource> queue = new ReferenceQueue<>(); PhantomReference<Resource>
phantom = new PhantomReference<>(resource, queue); // After GC finalizes the object, reference
appears in queue // Used by Cleaner API (NIO DirectByteBuffer cleanup) // Cleaner API (JDK 9+)
– preferred over finalize() Cleaner cleaner = Cleaner.create(); cleaner.register(resource, ()
-> nativeCleanup(nativeHandle));
```

5. G1GC tuning — MaxGCPauseMillis and region sizing

```
// G1GC key tuning parameters // -XX:MaxGCPauseMillis=200 (pause goal, default 200ms) // G1
adjusts young gen size to meet this target // Lower values => more frequent, smaller GCs //
Higher values => less frequent, larger GCs (more throughput) // -XX:G1HeapRegionSize=N{m} (1MB
to 32MB, must be power of 2) // Larger regions => fewer humongous object allocations // For 8GB
heap: 8*1024/2048 regions = 4MB region size // -XX:InitiatingHeapOccupancyPercent=45 (IHOP) //
% of heap at which concurrent marking starts // Lower => start marking earlier, fewer Full GCs
// Too low => wasted CPU on unnecessary marking cycles // -XX:G1ReservePercent=10 (emergency
reserve, default 10%) // -XX:ConcGCThreads=4 (concurrent marking threads) //
-XX:ParallelGCThreads=8 (STW GC threads) // Diagnose G1 with GCEasy (paste gc.log at gceasy.io)
// Look for: Evacuation Failures, humongous allocations, long mixed GC
```

6. DirectByteBuffer off-heap memory management

```
import java.nio.ByteBuffer; // Allocate off-heap DirectByteBuffer (not on GC-managed heap)
ByteBuffer direct = ByteBuffer.allocateDirect(1024 * 1024); // 1MB // Direct buffers are
cleaned by a Cleaner when the ByteBuffer // object is GC-collected, but GC may not run
promptly. // For explicit release: import sun.misc.Cleaner; // JDK 9+ safe reflection
alternative: if (direct instanceof sun.nio.ch.DirectBuffer) { ((sun.nio.ch.DirectBuffer)
direct).cleaner().clean(); } // Monitor direct memory usage // -XX:MaxDirectMemorySize=512m
(cap total direct memory) // MBean: java.nio:type=BufferPool,name=direct import
java.lang.management.ManagementFactory; import javax.management.*; MBeanServer mbs =
ManagementFactory.getPlatformMBeanServer(); ObjectName name = new
ObjectName("java.nio:type=BufferPool,name=direct"); long usedBytes = (Long)
mbs.getAttribute(name, "MemoryUsed"); long capacity = (Long) mbs.getAttribute(name,
"TotalCapacity"); System.out.printf("Direct memory: %d / %d bytes\n", usedBytes, capacity);
```

Interview Q&A;

Q1. What are the JVM memory regions and what lives in each?

Heap: Eden (new object allocation), Survivor S0/S1 (objects surviving Minor GC), Old Generation (tenured objects). Metaspace (native memory): class metadata, method bytecode, interned strings, static field values. Code Cache (native): JIT-compiled machine code. Off-Heap: DirectByteBuffer memory, memory-mapped files. Thread stacks are separate native memory per thread. Native method interface (JNI) memory is also outside the heap.

Q2. What triggers a Minor GC vs a Major/Full GC?

Minor (Young) GC: triggered when Eden space is full. Collects only Young Generation (Eden + Survivor spaces). Fast (milliseconds). Objects surviving enough Minor GCs are promoted to Old Gen. Major GC (G1 concurrent): triggered when Old Gen occupancy exceeds IHOP threshold. G1 runs concurrent marking then mixed GC collections. Full GC: stop-the-world collection of entire heap — triggered by promotion failure, Metaspace exhaustion, or explicit `System.gc()`. Full GCs cause the longest pauses.

Q3. How does G1GC differ from the Parallel GC?

G1GC divides the heap into equal-sized regions (1-32MB) rather than fixed Young/Old areas. It collects regions with the most garbage first (Garbage-First). G1 targets a configurable pause goal (`MaxGCPauseMillis`) by adjusting how many regions to collect per GC cycle. Parallel GC is a throughput collector that uses multiple threads for stop-the-world Young and Old collections with no pause goal. G1 trades some throughput for more predictable pauses, making it better for latency-sensitive services.

Q4. What is ZGC and what makes it different from G1GC?

ZGC (available production-ready from JDK 15) is a low-latency collector targeting sub-millisecond GC pauses even on multi-terabyte heaps. It uses colored pointers (metadata bits in 64-bit object references) and load barriers to perform concurrent object relocation — the most expensive phase of GC — without stopping application threads. G1GC still has stop-the-world evacuation phases. ZGC's trade-off is slightly higher CPU overhead and lower throughput compared to G1GC.

Q5. What are the types of `OutOfMemoryError` and what causes each?

`java.lang.OutOfMemoryError`: Java heap space — heap is full; live objects fill all of Eden, Survivor, and Old Gen. Fix: increase `-Xmx`, find retention leaks. Metaspace — class metadata exhausted native memory. Often caused by classloader leaks (dynamic code generation, hot deployment). Fix: `-XX:MaxMetaspaceSize`. GC overhead limit exceeded — JVM spends more than 98% of time in GC and recovers less than 2% of heap. Indicates near-total heap retention. Unable to create new native thread — OS thread limit exceeded.

Q6. How do you diagnose a memory leak in a Java application?

Step 1: Monitor heap usage over time (`/actuator/metrics/jvm.memory.used`) — a sawtooth pattern that never returns to baseline after GC indicates a leak. Step 2: Enable `-XX:+HeapDumpOnOutOfMemoryError`. Step 3: Capture a heap dump with `jmap -dump:live`. Step 4: Open in Eclipse MAT; run Leak Suspects report to find objects with unexpectedly high retained size. Step 5: Inspect the dominator tree and reference chains to identify the GC root holding the objects. Common roots: static collections, `ThreadLocals`, registered listeners.

Q7. What object retention patterns commonly cause memory leaks?

Static collections: List/Map fields that grow without bounds. Inner class references: non-static inner classes hold an implicit reference to the outer class instance. `ThreadLocal` variables: in thread pool environments, `ThreadLocals` persist for the thread's lifetime unless explicitly removed in finally blocks. Event listeners/observers: listeners registered with a long-lived publisher but never deregistered. Session/cache objects: unbounded session maps in web applications.

Q8. What is a dominator tree in heap analysis?

In a heap object graph, object A dominates object B if every path from GC roots to B passes through A. The dominator tree shows, for each object, the retained size — the bytes that would be freed if that object were collected. In Eclipse MAT, the dominator tree shows the top objects by retained size, revealing which objects are responsible for most heap usage. An entry with a very large retained size but small shallow size indicates it is an indirect root of a large object graph.

Q9. What is the difference between shallow size and retained size?

Shallow size is the memory consumed by the object itself — its fields, not the objects they reference. For a `HashMap` the shallow size is just the `HashMap` instance fields (load factor, size, array reference). Retained size is the total memory that would be freed if the object (and everything exclusively reachable through it) were collected. A `HashMap` retaining 10,000 `String` entries has small shallow size but potentially large retained size.

Q10. How do you tune G1GC MaxGCPauseMillis effectively?

Set `MaxGCPauseMillis` to your latency SLA (e.g., 200ms). G1 adjusts Young Gen size to meet this: smaller Young Gen => more frequent but shorter GCs; larger Young Gen => less frequent but longer. Also tune `InitiatingHeapOccupancyPercent` (default 45%) — lower triggers concurrent marking earlier, reducing Full GC risk but using more CPU. Increase `G1HeapRegionSize` (4-16MB) to reduce humongous allocations. Monitor GC logs for evacuation failures (insufficient Old Gen space) — add `-XX:G1ReservePercent` to increase emergency reserve.

Q11. What is a humongous object in G1GC?

A humongous object is an object larger than half a G1 region size. Humongous objects are allocated directly in Old Generation regions (bypassing Young Gen), which increases Old Gen occupancy and can trigger mixed GC collections prematurely. Large arrays (`byte[]`, `int[]`) are common culprits. Fix: increase `G1HeapRegionSize` (up to 32MB) so fewer objects qualify as humongous, or redesign the application to avoid large array allocations.

Q12. When should you use weak references vs soft references?

`WeakReference`: use when you want the reference to be collected at the next GC cycle when no strong references exist. Typical use: `WeakHashMap` for canonical mapping registries, listener registries where listeners may be GC-collected. `SoftReference`: use for memory-sensitive caches. The JVM guarantees soft references are cleared before throwing `OutOfMemoryError`. Items are kept as long as memory is available. `PhantomReference`: use for post-finalization native resource cleanup (preferred over `finalize()` since JDK 9 via Cleaner API).

Q13. How do you monitor GC activity in production?

Enable GC logging: `-Xlog:gc*:file=gc.log:time,uptime`. Monitor via Micrometer JVM metrics: `jvm.gc.pause` (p99 pause time), `jvm.gc.memory.promoted` (old gen promotion rate), `jvm.memory.used` (heap used). Alert when: GC pause p99 > SLA threshold, Old Gen usage approaches `Xmx`, allocation rate exceeds GC throughput (heap fills faster than it is collected). Use `GCEasy.io` or `GCViewer` to analyse GC log files offline after incidents.

Q14. What is the Code Cache and what happens when it fills?

The Code Cache is native memory holding JIT-compiled methods. When the Code Cache fills (`-XX:ReservedCodeCacheSize`, default 240MB in JDK 11+), the JVM prints "CodeCache is full" and stops JIT-compiling new methods. Existing compiled methods are deoptimised over time, forcing interpretation — causing significant CPU regression (2-10x slower). Increase with `-XX:ReservedCodeCacheSize=512m`. Monitor with `-XX:+PrintCodeCache` or `MBean java.lang:type=Compilation`.

Q15. How does DirectByteBuffer off-heap memory work?

`ByteBuffer.allocateDirect()` allocates native memory outside the JVM heap, accessible via the Unsafe API. It is not subject to GC pressure. The `DirectByteBuffer` Java object is on-heap; when it is GC-collected, a Cleaner (`PhantomReference`) frees the native memory. Explicit deallocation requires calling `cleaner.clean()`. Cap total direct memory with `-XX:MaxDirectMemorySize`. Monitor via JMX `MBean java.nio:type=BufferPool,name=direct`. Used by NIO, Netty, Kafka for zero-copy I/O.

Q16. What does "GC overhead limit exceeded" mean and how do you fix it?

This OOM is thrown when the JVM determines it spent more than 98% of elapsed time performing GC and recovered less than 2% of the heap across the last 5 Full GC cycles — indicating the application is essentially out of live heap. Root cause is almost always a memory leak or drastically undersized heap. Fix: (1) increase -Xmx; (2) capture a heap dump immediately before the error with -XX:+HeapDumpOnOutOfMemoryError; (3) analyse with MAT to find the retained object graph; (4) fix the retention leak.

Q17. How do you analyse GC logs?

GC logs (-Xlog:gc*:file=gc.log:time,uptime in JDK 11+) contain: GC type (Young/Mixed/Full), cause (Allocation Failure, Metadata GC Threshold), duration (pause time), heap before and after. Parse with GCEasy (web tool) or GCViewer (desktop). Key metrics: throughput (% time NOT in GC, target > 95%), max pause time (vs SLA), allocation rate (MB/s), promotion rate (MB/s to Old Gen), humongous allocation count, evacuation failure count.

Q18. What is metaspace and when does it run out?

Metaspace is native memory (introduced in JDK 8, replacing PermGen) holding class metadata: bytecode, method data, class hierarchy info, interned strings via String.intern(). It grows automatically by default (cap with -XX:MaxMetaspaceSize). It fills when: classloader leaks occur (e.g., dynamic code generation tools like Groovy, CGLib, ASM create many class loaders that are never unloaded); hot redeployment in app servers without proper classloader cleanup. Monitor with jvm.memory.used{area=nonheap} in Micrometer.

Q19. What is allocation profiling and how do you use it to reduce GC pressure?

Allocation profiling (async-profiler alloc mode, JFR ObjectAllocationInNewTLAB) records where objects are allocated by bytes. The flame graph shows the call sites responsible for the most heap allocation. High allocation sites increase Minor GC frequency. Optimizations: reuse objects (StringBuilder instead of + concatenation), use primitive arrays instead of boxed collections, pool expensive objects (HttpClient, ObjectMapper), avoid creating temporary objects in hot loops, use off-heap storage for large data buffers.

Q20. What is Shenandoah GC and how does it compare to ZGC?

Shenandoah is a low-latency concurrent GC developed by Red Hat, available in OpenJDK. Like ZGC it performs concurrent evacuation (object moving) without stopping application threads. Shenandoah uses Brooks forwarding pointers (an extra pointer per object) for concurrent relocation, whereas ZGC uses colored pointer bits. Both target sub-millisecond pauses. ZGC is designed to scale to TB-range heaps; Shenandoah excels for moderate heap sizes. Both have higher CPU overhead than G1GC for the same workload.